

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 11 月 6 日

浙江大学实验报告

课程名称：____计算机组成与设计____ 实验类型：____综合____

实验项目名称：____IP 核集成 SOC 设计____

学生姓名：____张序鸿____ 专业：____混合班____ 学号：3230103265

指导老师：____杨莹春、潘之杰____ 助教：____张立冬、马致远____

实验地点：____东 4-509____ 实验日期：2024 年 9 月 25 日

一、实验目的和要求

1. 初步了解 GPIO 接口与设备
2. 了解计算机系统的基本结构
3. 了解计算机各组成部分的关系
4. 了解并掌握 IP 核的使用方法
5. 了解 SOC 系统并用 IP 核实现简单的 SOC 系统

二、实验内容和原理

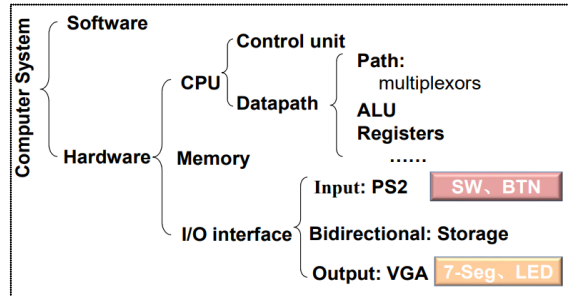
内容：

通过第三方 IP 和已有 IP 模块建立 CPU 测试环境（SOC 系统的集成实现）

-----参考原理图采用 Verilog 调用子模块的方式实现，完成由按键消抖、时钟分频、8 通道选择、数码管驱动、LED 驱动、VGA 液晶驱动以及存储 ROM、RAM 等模块构建的**实验平台**。

原理:

2.1 计算机主要组成结构



本实验主要任务是理解 Computer System 中硬件部分的作用及相互之间的关系，从而能够将 CPU、Memory 储存、I/O 接口等模块进行连接，从而构建一个能够对 CPU 进行简单操作的实验平台。

该 SOC 系统主要由 11 个子模块构成：

U1:	CPU	-SCPU
U2:	ROM	-ROM_D
U3:	RAM	-RAM_B
U4:	总线(含外设3~4)	-MIO_BUS
U5:	七段显示接口	-Multi_8CH32
U6:	外设1- 七段显示设备	-Seg7_Dev
U7:	外设2-GPIO接口及LED	-PIO
U8:	辅助模块一，通用分频模块	-Clk_div
U9:	辅助模块二，机械去抖模块	-Anti_jitter
U10:	通用计数器	-Counter_x
U11:	VGA显示接口	-VGA

2.2 SOC 系统核心介绍

2.2.1 CPU 模块：SCPU

该 CPU 采用 RISC-V 架构，需要设计实现 R、I、S、B、U、J-Type 中的若干基本指令，同时包括了以下功能：

- 指令控制：CPU 接收输入的指令 `inst_in[31:0]`并按程序逻辑顺序 产生下一条指令地址 `PC_out[31:0]`；所以 CPU 中应该包含指令 指针寄存器(PC)和地址计算单元。

- 操作控制：CPU 接收指令后，能产生指令执行过程要求的、指 令功能约定的微操作控制信号；所以 CPU 中应该包含指令译码 单元和微操作控制信号的产生单元。

- 数据加工：CPU 按照指令的要求，对输入的数据 `Data_in[31:0]`进行相应的算数运算、逻辑运算；左移、右移等移位操作.....后输出结果数据 `Data_out[31:0]`。

- 外部访问：CPU 能够进行存储器和 I/O 访问，通过地址输出 `Addr_out[31:0]`能够访问到外

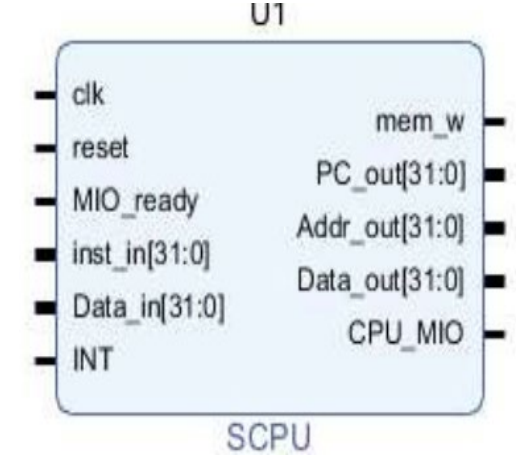
部存储区域。

- 中断处理: CPU 能够处理外部的中断和内部的异常情况, 所以应该包含中断控制处理单元, 通过 INT 输入来进行中断操作。(在后续实验中会添加, Lab2 不涉及【LAB4】)

- 时序控制: CPU 能够提供对所有控制信号的定时控制, 包含时序信号产生电路。

CPU 主要由 DataPath 、 Control unit 两部分组成, 后续实验需要分别实现、拓展、组装这两部分, 完成更多操作。

```
SCPU U1(  
    .Addr_out(addr),  
    .Data_in(Cpu_data4bus),  
    .Data_out(Cpu_data2bus),  
    .MIO_ready(1'b0),  
    .MemRW(mem_w),  
    .PC_out(PC_out[31:0]),  
    .clk(clk_CPU),  
    .inst_in(Inst_in),  
    .rst(rst)  
);
```



2.3 SOC 系统存储介绍

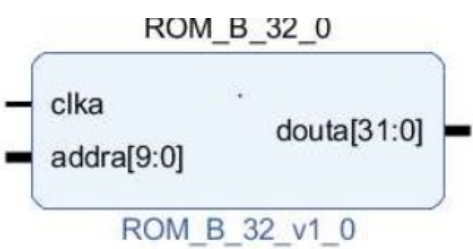
2.3.1 指令代码存储模块: ROM_B

该模块由 FPGA 内部储存器中的 Distributed Memory Generator (ROM 类型, 无 clka 引脚) 生成, 容量为 1024 * 32bit, 初始化文档所使用的 .coe 在每个实验中均会微调, 本实验使用 I_mem.coe (斐波那契数列求和, 检查 ALU 结果)。

PC	Machine Code	Basic Code	Original Code	Result
0x0	0x00100093	addi x1 x0 1	loop:addi x1,x0,1	#x1=00000001
0x4	0x00102133	slt x2 x0 x1	slt x2,x0,x1	#x2=00000001
0x8	0x002101B3	add x3 x2 x2	add x3,x2,x2	#x3=00000002
0xc	0x00218233	add x4 x3 x2	add x4,x3,x2	#x4=00000003
0x10	0x003202B3	add x5 x4 x3	add x5,x4,x3	#x5=00000005

因此作为只读存储器, 它在本系统中作为指令代码的存储部件, 输入地址来自 CPU 的 PC 指针输出: PC_out, 然后在时钟信号的作用下输出对应的指令信息作为 CPU 的指令输入 inst[31:0]。

```
Rom_D_0 U2 (  
    .a(PC_out[11:2]),  
    .spo(Inst_in) );
```

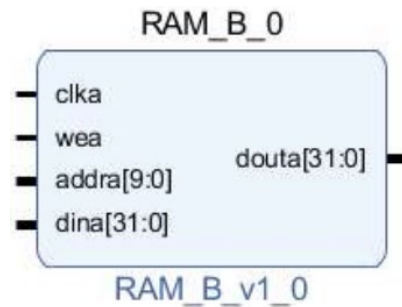


2.3.2 数据存储模块：RAM_B

该模块由 FPGA 内部储存器中的 Block Memory Generator (RAM 类型，有 clka 引脚) 生成，容量为 $1024 * 32\text{bit}$ 。但本实验中，模块采用已给的 .edf 和 .v 文件，不用个人生成。

作为 CPU 应用的外部数据存储部件的随机存储器，支持读写操作（通过 wea 信号势能），其中的数据线均来自总线 MIO-BUS。

```
RAM_B U3(  
    .addra(ram_addr), // 地址线  
    .clka(~clk_100mhz), // 与 CPU 反向  
    .dina(ram_data_in), // 输入数据线  
    .douta(RAM_B_douta), // 输出数据线  
    .wea(data_ram_we) // 读写使能信号  
);
```



2.4 SOC 系统总线介绍

2.4.1 总线接口模块：MIO-BUS

总线（**MIO-BUS**）作为 SOC 中各个功能部件之间传送信息的公共通道，按功能可以分为数据总线、地址总线、控制总线。本实验平台中，它为 CPU、数据存储部件、外设 IO 接口设备（SW 开关、按键 BTN、显示设备 LED 等）提供了数据交互的通道，从而保证 CPU 能够完成相应的操作。

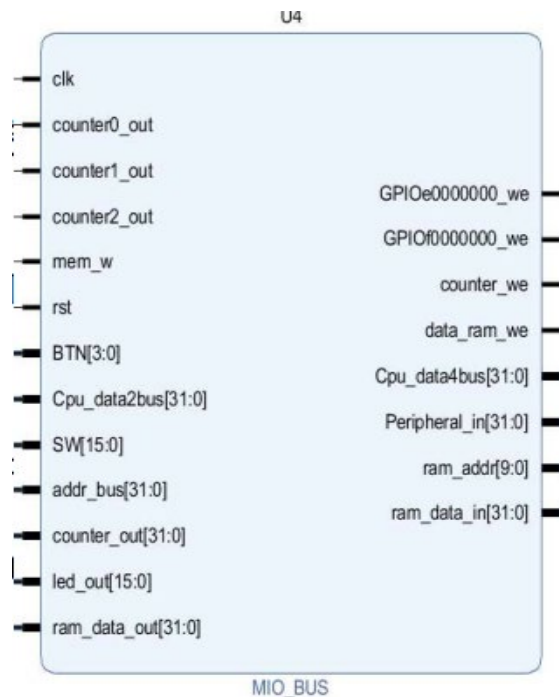
本实验的 SOC 系统接口分为两种：**Master**（主设备）主动发起读写操作；**Slave**（从设备）被动响应读写操作。这样的结构简化了硬件设计和系统结构，但是因为一般来说一个 **Master** 只能与一个 **Slave** 进行交互，造成了性能瓶颈。

```
MIO_BUS U4(  
    .clk(clk_100mhz),           // 主板时钟  
    .rst(rst),  
    .BTN(BTN_OK[3:0]),  
    .SW(SW_OK[15:0]),  
    .mem_w(mem_w),              // 存储器读写操作  
    .Cpu_data2bus(Cpu_data2bus[31:0]), // CPU 输出数据总线  
    .addr_bus(addr[31:0]),       // 地址总线  
    .ram_data_out(RAM_B_douta[31:0]),  
    .led_out(LED_out[15:0]),  
    .counter_out(counter_out[31:0]),  
    .counter0_out(counter0_out),  
    .counter1_out(counter1_out),
```

```

        .counter2_out(counter2_out),
        .Cpu_data4bus(Cpu_data4bus[31:0]), // CPU 写入数据总线
        .ram_data_in(ram_data_in[31:0]),
        .ram_addr(ram_addr[9:0]),
        .data_ram_we(data_ram_we),
        .GPIOf0000000_we(GPIOf0000000_we),
        .GPIOe0000000_we(GPIOe0000000_we),
        .counter_we(counter_we),
        .Peripheral_in(Peripheral_in[31:0])
    );

```



2.5 外设 GPIO 接口设备介绍

2.5.1 GPIO 设备 (1) --SPIO

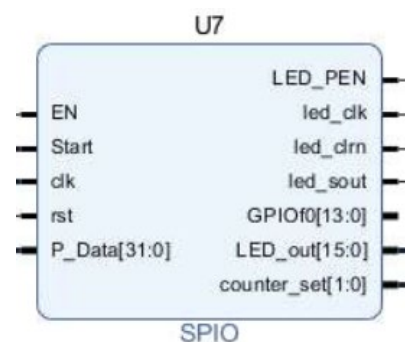
SPIO 作为本实验中的输出设备之一，用以 LED 显示部件的驱动和计数器 Counter 的通道控制，从而能够进行显示调试。

读写控制信号：GPIOf0000000_we(GPIOffffff00_we)，来自 MIO-BUS。

```

SPIO U7(
    .clk(~clk_CPU),
    .rst(rst),
    .Start(clk_div[20]),
    .EN(GPIOf0000000_we),
    .P_Data(Peripheral_in),
    .counter_set(counter_set),
    .LED_out(LED_out)
);

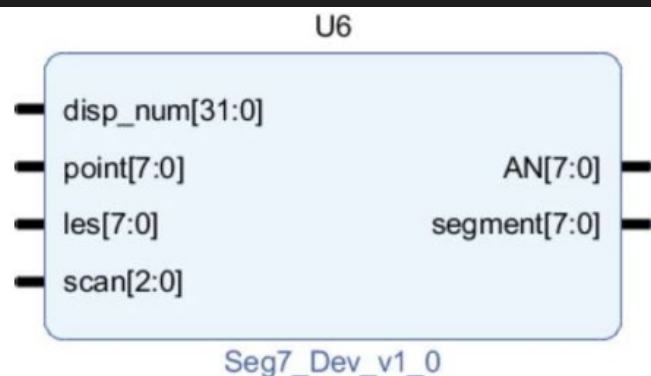
```



2.5.2 GPIO 设备 (2) -- Seg7_Dev

本质上就是一个简单的七段数码管的显示驱动，输入来自 8 通道选通 **Multi_8CH32** 输出的 8 位小数点、8 位七段码使能、动态刷新频率、32 位待显示数据。

```
Seg7_Dev_0 U6(  
    .disp_num(disp_num),  
    .point(point_out),  
    .les(le_out),  
    .scan({clk_div[18],clk_div[17],clk_div[16]}),  
    .AN(AN),  
    .segment(segment)  
);
```

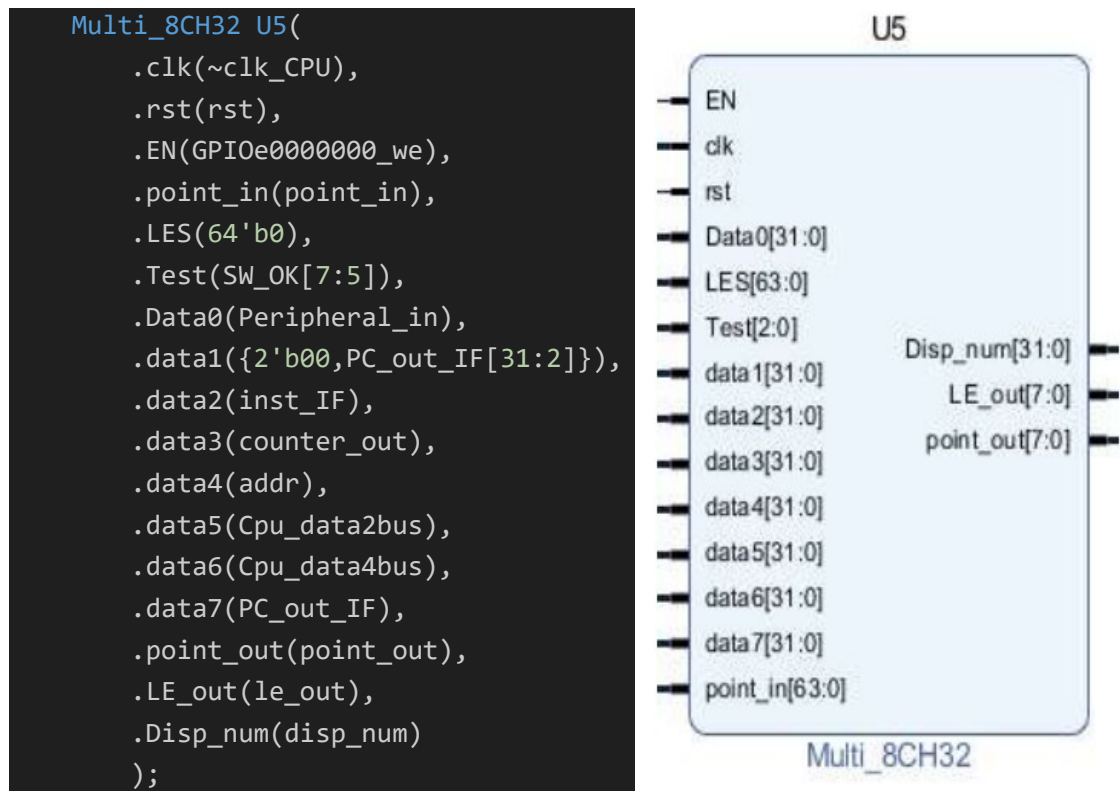


2.5.3 GPIO 接口 (2) -- Multi_8CH32

该 8 路数据通路模块用于控制 Seg7_Dev 模块，控制信号 Test = SW[7:5] 000-111 共 8 种值，从而选择通道 data [0-7]:

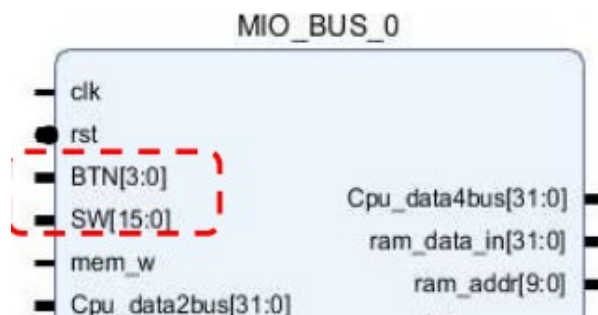
SW[7:5]=000; 输出Data0;数据源为总线的Peripheral_in, 源头是Cpu_data2bus
SW[7:5]=001; 输出data1;数据源为CPU指令字节地址PC_out[31:2]
SW[7:5]=010; 输出data2;数据源为指令存储器数据输出Inst_in (31:0)
SW[7:5]=011; 输出data3;数据源为计数器输出Counter_out(31:0)
SW[7:5]=100; 输出data4;数据源为数据存储器地址输出addr_out (31:0)
SW[7:5]=101; 输出data5;数据源为Cpu_data2bus,源头是CPU数据输出Data_out
SW[7:5]=110; 输出data6;数据源为CPU数据输入Cpu_data4bus, 源头是数据存储器数据输出ram_data_out [31:0]
SW[7:5]=111; 输出data7;数据源为CPU指令字节地址PC_out[31:0]

选择合适的数据递交给 Disp_num,同时根据外部输入的 point_in 以及 LES 来控制 8 个七段数码管的小数点显示和闪烁控制。



2.5.4 GPIO 设备（3 & 4）-- SW & BTN

8 位 Switch 和 4 位 Button 输入设备，由于比较简单，两者都被包括在了 MIO-BUS 的模块中。



2.5.5 GPIO 设备（5）-- Counter

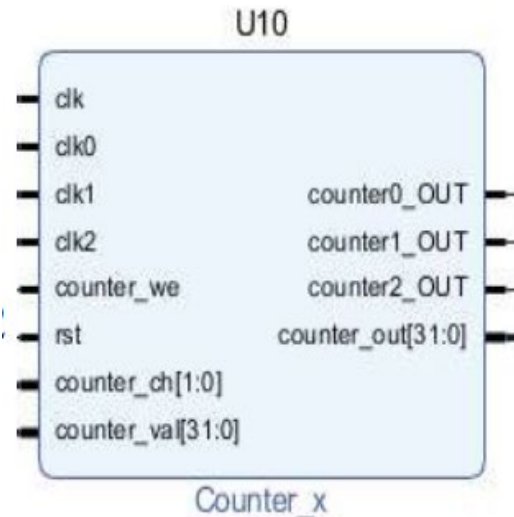
三通道独立计数器，可用于程序定时。

其中 counter_ch 进行计数器通道控制，clk0、clk1、clk2 分别对应的时钟分频为 clk_div[7]、clk_div[9]、clk_div[11]。


```

Counter_x U10(
    .clk(~clk_CPU),
    .rst(rst),
    .clk0(clk_div[6]),
    .clk1(clk_div[9]),
    .clk2(clk_div[11]),
    .counter0_OUT(counter0_out),
    .counter1_OUT(counter1_out),
    .counter2_OUT(counter2_out),
    .counter_we(counter_we),
    .counter_out(counter_out),
    .counter_val(Peripheral_in),
    .counter_ch(counter_set)
);

```



2.5.6 GPIO 显示调试设备（6）-- VGA

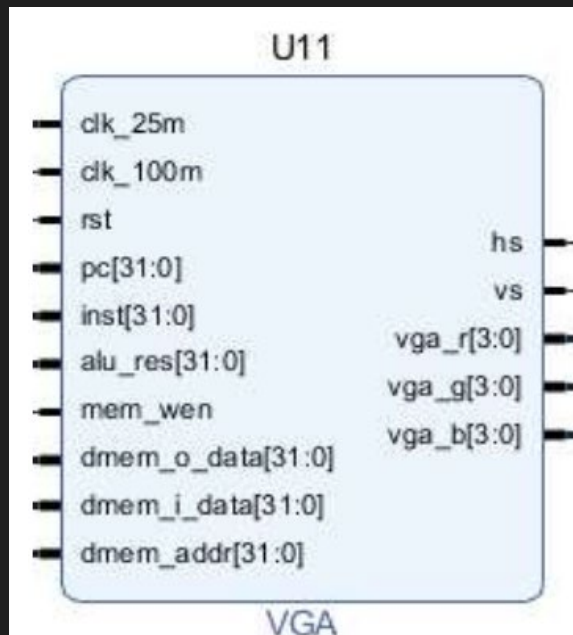
VGA 显示设备主要由 3 个子模块组成(Lab2 给出了网表文件因此不用组装):
Vga_controller （水平、垂直扫描控制, 25mhz 时钟）、Vga_display （接受像素信息, 输出 RGB 驱动信号, 100mhz 时钟）、Vga_debugger （显示信息输入及格式转换）。

本模块主要将 CPU 调试信息呈现在液晶屏上, 便于验证及修改代码。

```

VGA U11(
    .clk_25m(clk_div[1]),
    .clk_100m(clk_100mhz),
    .rst(rst),
    .PC_IF(PC_out_IF[31:0]),
    .PC_ID(PC_out_ID[31:0]),
    .inst_IF(inst_IF),
    .inst_ID(inst_ID),
    .PC_Ex(PC_out_Ex),
    .MemRW_Ex(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .hs(HSYNC),
    .vs(VSYNC),
    .vga_r(Red),
    .vga_g(Green),
    .vga_b(Blue)
);

```



2.6 系统辅助模块介绍

2.6.1 通用分频模块 clk_div

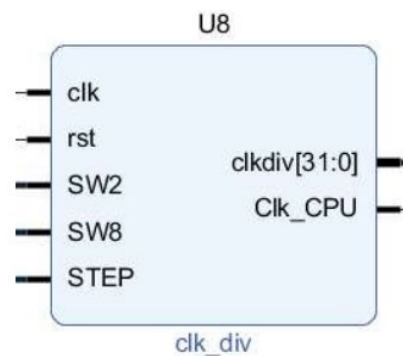
在逻辑实验中已经实现了较简单的版本，本实验进行改造通过添加 SW[2][8] 来得到 3 种不同的时钟：

SW[2][8] = 00 --> 输出时钟为全速（100mhz）

SW[2][8] = 10 --> 输出时钟为自动单步频率（clkdiv[24]）

SW[2][8] = X1 --> 输出时钟为手动单步频率（利用 STEP = SW[10]控制）

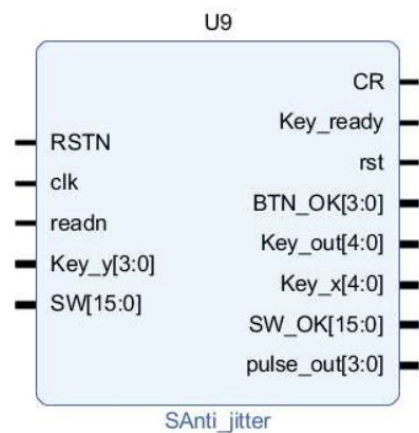
```
clk_div U8(  
    .clk(clk_100mhz),  
    .rst(rst),  
    .SW2(SW_OK[2]),  
    .SW8(SW_OK[8]),  
    .STEP(SW_OK[10]),  
    .clkdiv(clk_div),  
    .Clk_CPU(clk_CPU)  
);
```



2.6.2 开关去抖动模块 Santi_jitter

根据输入的机械开关量，消除开关和按钮的机械抖动后，输出滤除抖动的逻辑值：SW -> SW_OK, BTN -> BTN_OK。

```
SAnti_jitter U9(  
    .clk(clk_100mhz),  
    .RSTN(RSTN),  
    .Key_y(BTN_y),  
    .SW(SW[15:0]),  
    .BTN_OK(BTN_OK[3:0]),  
    .SW_OK(SW_OK[15:0]),  
    .rst(rst)  
);
```



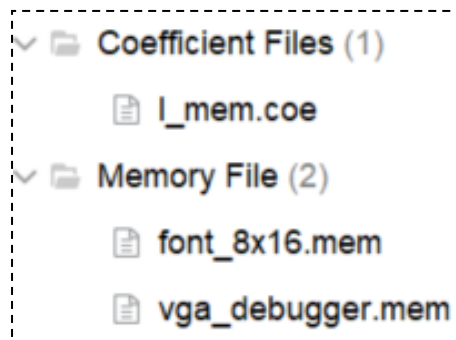
三、实验过程和数据记录

3.1 建立 SOC 应用工程：

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp02-IP2SOC ；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1；

Lab2 不需要自己实现子模块，只需要导入对应的 EDF 文件或利用 IP Catalog (Seg7_Dev) 添加调用，其中 ROM 模块需要导入对应的.coe 文件。
同时导入用于 VGA 静态显示的初始化信息文件 (.mem) 到工程。



(2) 结构化描述设计 SOC：

- 创建 TOP 文件 ： CSSTE.v

- 数据接口如下：

```
module CSSTE(  
    input clk_100mhz,  
    input RSTN,  
    input [3:0] BTN_y,  
    input [15:0] SW,  
    output [3:0] Blue,  
    output [3:0] Green,  
    output [3:0] Red,  
    output HSYNC,  
    output VSYNC,  
    output [15:0] LED_out,  
    output [7:0] segment,  
    output [7:0] AN  
);
```

- 逻辑实现：

(1) 中间传递变量:

```
wire [3:0] BTN_OK;
wire [15:0] SW_OK;
wire rst;
wire mem_w;
wire [31:0] Cpu_data2bus;
wire [31:0] addr;
wire [31:0] ram_data_out;
wire [31:0] counter_out;
wire counter0_out;
wire counter1_out;
wire counter2_out;
wire [31:0] Cpu_data4bus;
wire [31:0] ram_data_in;
wire [9:0] ram_addr;
wire data_ram_we;
wire GPIOf0000000_we;
wire GPIOe0000000_we;
wire counter_we;
wire [31:0] Peripheral_in;

wire [31:0] clk_div;
wire clk_CPU;

wire [1:0] counter_set;
wire [63:0] point_in;
assign point_in = {clk_div,clk_div};

wire [31:0] inst_IF;
wire [31:0] PC_out_IF , PC_out_ID , PC_out_Ex, inst_ID , Data_out_WB;
wire MemRW_EX , MemRW_Mem ;

wire [7:0] point_out;
wire [7:0] le_out;
wire [31:0] disp_num;

wire [31:0] RAM_B_douta;
```

(2) 具体子模块调用：

```
SCPU U1();

Rom_D_0 U2 ();

RAM_B U3();

MIO_BUS U4();

Multi_8CH32 U5();

Seg7_Dev_0 U6();

SPIO U7();

clk_div U8();

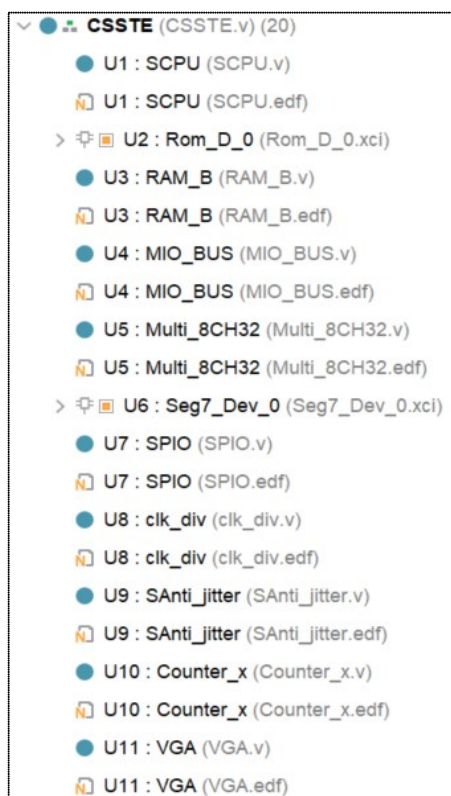
SAnti_jitter U9();

Counter_x U10() ;

VGA U11();
```

其中具体代码已经在实验原理部分给出，此处就不再多加。

(3) 工程层次结构：



（4）CSSTE 模块验证：

由于本实验均由 IP 固核组成，无法进行时序仿真，因此使用下载.bit 文件，使用 DEMO 程序进行物理验证。

本实验物理验证操作时主要注意以下几个部分：

- Seg7 显示通道选择（SW[7:5]控制）：

SW[7:5]=000:	CPU程序运行输出
SW[7:5]=001:	测试PC字地址
SW[7:5]=010:	测试指令字
SW[7:5]=011:	测试计数器
SW[7:5]=100:	测试RAM地址
SW[7:5]=101:	测试CPU数据输出
SW[7:5]=110:	测试CPU数据输入

其中注意：PC 字地址、指令字以及 RAM 地址，`dmem_addr = alu_res`，因此可以起到 ALU 结果显示的作用。

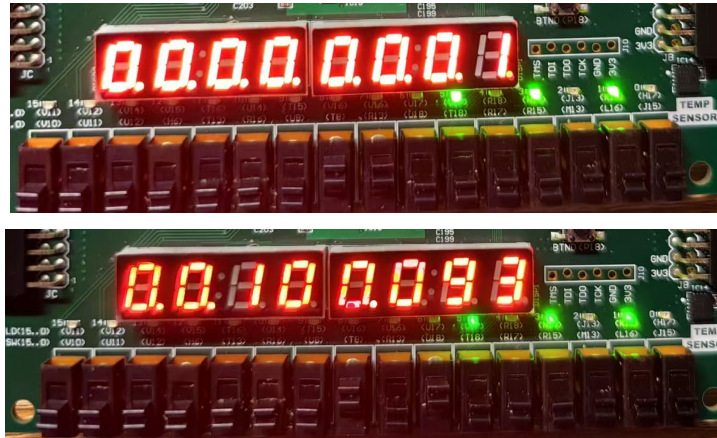
- 时钟信号调节（SW[8]SW[2]）：

开关	位置	功能
SW[8]SW[2]	00	CPU全速时钟 100MHZ
SW[8]SW[2]	01	CPU自动单步时钟(2*24分频)
SW[8]SW[2]	1X	CPU手动单步时钟
SW[10]	0~1	CPU手动单步时钟(开关SW[10]从0到1)

因为人眼的问题，我们只能调到 SW[8]SW[2] = 1X 的状态进行手动单步。

- ROM 中填充指令（I_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
00100093,00102133,002101B3,00218233,003202B3,00428333,
005303B3,00638433,007404B3,00848533,009505B3,00A58633,
00B606B3,00C68733,00D707B3,00E78833,00F808B3,01088933,
011909B3,01298A33,013A0AB3,014A8B33,015B0BB3,016B8C33,
017C0CB3,018C8D33,019D0DB3,01AD8E33,01BE0EB3,01CE8F33,
01DF0FB3,F80002E3;
```

```

RV32I Single Cycle CPU
pc: 00000000 inst: 00100093

x0: 00000000 ra: 00000000 sp: 00000000 gp: 00000000 tp: 00000000
t0: 00000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 00000001 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: 00000000 dmem_i_data: 00000000 dmem_addr: 00000001

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000

```

addi x1, x0, 1 - -> 0x00100093



```

RV32I Single Cycle CPU
pc: 00000004 inst: 00102133

x0: 00000000 ra: 00000000 sp: 00000000 gp: 00000000 tp: 00000000
t0: 00000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 00000001 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: 00000000 dmem_i_data: 00000001 dmem_addr: 00000001

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000

```

slt x2, x0, x1 - -> 0x00102133



```
RV32I Single Cycle CPU
pc: 00000008  inst: 002101b3

x0: 00000000  ra: 00000000  sp: 00000000  gp: 00000000  tp: 00000000
t0: 00000000  t1: 00000000  t2: 00000000  s0: 00000000  s1: 00000000
a0: 00000000  a1: 00000000  a2: 00000000  a3: 00000000  a4: 00000000
a5: 00000000  a6: 00000000  a7: 00000000  s2: 00000000  s3: 00000000
s4: 00000000  s5: 00000000  s6: 00000000  s7: 00000000  s8: 00000000
s9: 00000000  s10: 00000000  s11: 00000000  t3: 00000000  t4: 00000000
t5: 00000000  t6: 00000000

rs1: 00  rs1_val: 00000000
rs2: 00  rs2_val: 00000000
rd: 00  reg_i_data: 00000000  reg_wen: 0

is_imm: 0  is_auiopc: 0  is_lui: 0  imm: 00000000
a_val: 00000000  b_val: 00000000  alu_ctrl: 0  cmp_ctrl: 0
alu_res: 00000002  cmp_res: 0

is_branch: 0  is_jal: 0  is_jalr: 0
do_branch: 0  pc_branch: 00000000

mem_wen: 0  mem_ren: 0
dmem_o_data: 00000000  dmem_i_data: 00000001  dmem_addr: 00000002

csr_wen: 0  csr_ind: 000  csr_ctrl: 0  csr_r_data: 00000000
mstatus: 00000000  mcause: 00000000  mepc: 00000000  mtval: 00000000
mtvec: 00000000  mie: 00000000  mip: 00000000
```

add x3, x2, x2 --> 0x002101b3



```
RV32I Single Cycle CPU
pc: 0000000c  inst: 00218233

x0: 00000000  ra: 00000000  sp: 00000000  gp: 00000000  tp: 00000000
t0: 00000000  t1: 00000000  t2: 00000000  s0: 00000000  s1: 00000000
a0: 00000000  a1: 00000000  a2: 00000000  a3: 00000000  a4: 00000000
a5: 00000000  a6: 00000000  a7: 00000000  s2: 00000000  s3: 00000000
s4: 00000000  s5: 00000000  s6: 00000000  s7: 00000000  s8: 00000000
s9: 00000000  s10: 00000000  s11: 00000000  t3: 00000000  t4: 00000000
t5: 00000000  t6: 00000000

rs1: 00  rs1_val: 00000000
rs2: 00  rs2_val: 00000000
rd: 00  reg_i_data: 00000000  reg_wen: 0

is_imm: 0  is_auiopc: 0  is_lui: 0  imm: 00000000
a_val: 00000000  b_val: 00000000  alu_ctrl: 0  cmp_ctrl: 0
alu_res: 00000003  cmp_res: 0

is_branch: 0  is_jal: 0  is_jalr: 0
do_branch: 0  pc_branch: 00000000

mem_wen: 0  mem_ren: 0
dmem_o_data: 00000000  dmem_i_data: 00000001  dmem_addr: 00000003

csr_wen: 0  csr_ind: 000  csr_ctrl: 0  csr_r_data: 00000000
mstatus: 00000000  mcause: 00000000  mepc: 00000000  mtval: 00000000
mtvec: 00000000  mie: 00000000  mip: 00000000
```

add x4, x3, x2 --> 0x00218233

四、实验结果分析

实验演示照片过多，并未在报告中全部给出（结果包是正确的），给出的部分包括了 RAM 输入地址（SW[7:5] = 100）、ROM 指令输出（SW[7:5] = 010）、PC 字地址（未给出，但在 VGA 显示中可以查看）。

为了方便进行结果分析，在 Lab2 报告中提前透支 Lab4 的实验照片（×）。

```
RV32I Single Cycle CPU
pc: 0000007c    inst: f80002e3

x0: 00000000    ra: 00000001    sp: 00000001    gp: 00000002    tp: 00000003
t0: 00000005    t1: 00000008    t2: 0000000d    s0: 00000015    s1: 00000022
a0: 00000037    a1: 00000059    a2: 00000090    a3: 000000e9    a4: 00000179
a5: 00000262    a6: 000003db    a7: 0000063d    s2: 00000a18    s3: 00001055
s4: 00001a6d    s5: 00002ac2    s6: 0000452f    s7: 00006ff1    s8: 0000b520
s9: 00012511    s10: 0001da31    s11: 0002ff42    t3: 0004d973    t4: 0007d8b5
t5: 000cb228    t6: 00148add

rs1: 00    rs1_val: 00000000
rs2: 00    rs2_val: 00000000
rd: 00    reg_i_data: 00000000    reg_wen: 0

is_imm: 0    is_auiopc: 0    is_lui: 0    imm: 00000000
a_val: 00000000    b_val: 00000000    alu_ctrl: 0    cmp_ctrl: 0
alu_res: 00000000    cmp_res: 0

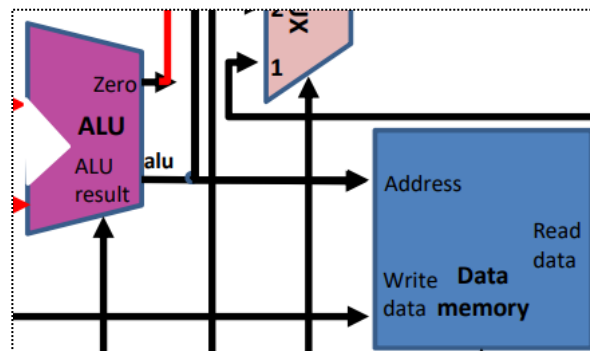
is_branch: 0    is_jal: 0    is_jalr: 0
do_branch: 0    pc_branch: 00000000

mem_wen: 0    mem_ren: 0
dmem_o_data: 00000000    dmem_i_data: 00000000    dmem_addr: 00000000

csr_wen: 0    csr_ind: 000    csr_ctrl: 0    csr_r_data: 00000000
mstatus: 00000000    mcause: 00000000    mepc: 00000000    mtval: 00000000
mtvec: 00000000    mie: 00000000    mip: 00000000
```

首先，可以比较简单地看出该 ROM 提供的指令是为了实现一个斐波那契数列（ $X_1 = 1$, $X_2 = 1$, $X_3 = 2$, $\dots$, $X_n = X_{n-1} + X_{n-2}$ ）。

- pc: 指令对应地址
- inst: 指令内容（机器码）
- x0, ra, sp, gp,: 各寄存器及对应值
- alu_res : ALU 计算结果
- dmem_addr : RAM 输入地址



（上图比较清晰地给出了 alu_res 和 dmem_addr 的关系）

五、讨论与心得

VGA 显示的内容中还有较大一部分未在本实验中调用，但通过其信息可以得到该指令的内容以及执行该指令下 SCPU_ctrl 中各控制信号的数值。

总而言之，本实验主打一个量大活细的连线过程，需要仔细对照。后续 Lab4 仍会用到 CSSTE.v 文件（修改 SCPU、VGA 部分），因此连线的工作还是比较重要的。

同时，本实验也能够比较直接地体验 CPU 的调试过程以及了解了 VGA 中各个信号的功能，可以为后续实验的代码 debug 提供基本的操作经验。

*吐槽：

0x38	0x00D707B3	add x15 x14 x13	add x15,x14,x13	#x15=00000262
0x3c	0x00E78833	add x16 x15 x14	add x16,x15,x14	#x16=000003DB
0x40	0x00F808B3	add x17 x16 x15	add x17,x16,x15	#x17=000006D3

（可恶的 I_mem.pdf，明明 $0x262+0x3db = 0x63d$ ，但却写了 $0x6d3$ ，白费了不少 debug 时间。。。）